Byzantine Fault Tolerant System Design for Dynamic Spectrum Sharing

Anonymous submission

***Abstract*—Dynamic spectrum sharing between incumbents and new entrants increasingly requires near-real-time, partially decentralized coordination to balance critical and lower-priority uses. This makes it important to reason about the robustness of spectrum sharing systems to malicious behavior.**

We design a spectrum sharing coordination system based on Byzantine Fault Tolerant (BFT) State Machine Replication (SMR) and evaluate its robustness to malicious behavior via simulation. We focus on a setting where transmissions of high-priority Type 1 users are detected via sensors (rather than explicit requests). Our results show that robustness to malicious behavior of sensors and of the coordination system servers depends on both the number of malicious nodes the system is configured to tolerate, and the physical system characteristics (i.e. number of sensors and their range). We further highlight the trade-off between minimizing interference for Type 1 users and maximizing spectrum usage, and its implications for system configuration.

I. Introduction

There is an increasing need for dynamic spectrum sharing based on near-real-time coordination to address new radio spectrum needs [1]. Ideally, there is little/no increased interference to incumbent uses of spectrum, while new uses can take advantage of idle spectrum in time and space. Shared spectrum is expected to support a combination of critical use cases (typically federal or other incumbents) and lower priority users (new entrants) that can opportunistically access idle spectrum. For example, in Citizens Broadband Radio Service (CBRS), US Navy radars must be able to operate without harmful interference, while new use cases such as private 5G networks are also being deployed. Systems for coordinating spectrum allocation will be distributed for fault tolerance, and are likely to be implemented in a decentralized manner, as there is not an obvious centralized organization to deploy such services. In CBRS, access is coordinated through a Spectrum Access System (SAS) deployed by multiple companies (SAS administrators [2]). In a shared spectrum environment, there is a need to consider the robustness of spectrum management systems to security breaches and malicious participants [1].

While blockchains have been proposed as a potential mechanism for coordinating spectrum access in the presence of malicious participants [3], [4], existing work largely assumes that spectrum sharing systems can be built on top of existing blockchain infrastructure, without clearly specifying how that infrastructure needs to be deployed and what properties it needs in order to support the promised resilience in practice.

In this paper, we present a concrete design for a spectrum sharing coordination system based on Byzantine Fault Tolerant (BFT) State Machine Replication (SMR) and evaluate its robustness to different types of malicious behavior under different deployment options via simulation. Our system is loosely inspired by CBRS and includes two tiers of users. Type 1 users have priority and may transmit at any time. Their transmissions are detected via *sensors*, similar to the environmental sensing capability (ESC) to detect US Navy radar signals in CBRS. Type 2 users have lower priority and make requests to access unused spectrum. They may only transmit if their requests are granted. Servers maintain a replicated database of ongoing transmissions by using a BFT SMR protocol to process all sensor reports and user requests and use this information to grant or deny user requests.

Our threat model includes both Byzantine servers and Byzantine sensors. A key contribution of our work is to analyze how the specific threat model and sensor deployment strategy affect the number of sensors needed to adequately protect Type 1 users from interference. We show that Byzantine server behavior beyond the tolerated threshold can make the system nearly unusable, highlighting the need for resilient solutions. In contrast, the system is relatively robust to sensor misbehavior, *when sensor density is sufficiently high*. The number of sensors needed to reach sufficient density depends on the deployment model and configured number of tolerated faulty sensors. We show that sharing sensor data between organizations (global deployment model) reduces the overall cost (number of required sensors). We further show that while increasing the tolerated number of faulty sensors reduces vulnerability to falsified reports, it increases vulnerability to maliciously omitted reports. Thus, an ideal configuration requires carefully considering application requirements (i.e., is it more important to use spectrum efficiently or avoid interference to incumbents?).

The specific contributions of our work are:

1) We design a BFT coordination system for dynamic spectrum sharing.
2) We evaluate via simulation: the impact of Byzantine server behavior, the number of sensors needed to mask Byzantine sensor behavior, and the application-level impacts of sensor misbehavior.
3) We provide design insights including: the benefit of sharing sensor data across organizations, and the need to consider application sensitivity to different types of sensor misbehavior in configuring the system.

II. Related Work

A. Spectrum Sharing

Spectrum has been largely allocated for "uses" such as data transfer (both terrestrial and satellite, fixed and mobile),

radiolocation (for example GPS), and sensing (both active and passive). Increasing demand for spectrum in these and other uses has made it necessary to share spectrum in innovative ways. Typically, such sharing involves coordination, on various time scales. One example is CBRS that has three tiers – the incumbent navy radars, licensed allocation, and unlicensed use with registration. Recently, coordination of spectrum between different assignments is being considered in the 37 GHz bands [5]. Decentralized coordination using automated frequency coordination (AFC) [6] is required in the 6 GHz bands (with WiFi) when access points use standard power, so that interference to incumbent microwave links is limited.

In general, as more previously allocated and assigned spectrum is opened up by regulatory authorities for sharing, coordination between incumbents and new entrants becomes inevitable. Incumbents, in our work, are referred to as Type 1 users and new entrants or secondary users are referred to as Type 2 users.

B. Blockchains for Spectrum Management

Prior work has identified blockchains a tool for spectrum management [3], which has led to a large number of blockchain-based spectrum management systems being proposed (e.g. [7]–[13], surveyed in [4], [14]). The most relevant works in this space are BD-SAS [7] and TrustSAS [8]. BD-SAS develops a CBRS-compatible blockchain-based system, where a global chain is used for regulatory actions, and local chains are used for allocating spectrum to users within a particular geographic region. Local chains (which have a similar function to our system) tolerate up to 50% Byzantine execution servers, but rely on local witnesses for block ordering and assume all witnesses behave correctly. It models environmental sensing reports as fully trusted and distributed by a trusted regulator. Their focus is on backwards compatibility with CBRS, handling an adaptive adversary in local chains, and providing acceptable throughput and latency performance.

TrustSAS similarly targets CBRS, but focuses on protecting the privacy of secondary (Type 2) users. It employs a hierarchical approach where *clusters* of secondary users obtain spectrum allocations via a global blockchain, and then make individual spectrum grants within each cluster using BFT consensus. It does not consider sensing, but rather assumes servers have consistent and up-to-date information about primary (Type 1) user transmissions.

In general, this line of work focuses on building spectrum allocation systems for specific requirements on top of existing blockchain systems and ignores sensing-related issues. In contrast, we analyze the fundamental BFT mechanisms needed for such a system, present a concrete design, and analyze the impact of configuration choices and Byzantine behavior on the system's ability to make effective allocation decisions.

C. BFT Spectrum Sensing

Byzantine sensor behavior has been investigated in spectrum sensing for cognitive radio (surveyed in [15]). In this case, the typical model is that there are a number of cognitive radios (CRs) that report sensing data to cooperatively determine whether or not a primary user is active, and each CR has some probability of being Byzantine. Existing work develops data fusion schemes that can (with high probability) accurately detect primary user presence/absence in this environment.

Some works consider models closer to ours, where a fixed number of nodes may be Byzantine. This includes generic classical fault-tolerant sensor averaging [16] and its privacy-preserving variants [17]. Work in the low-earth-orbit satellite context uses exact and approximate variants of Byzantine Agreement to allow operators with noisy analog measurements to reach agreement on spectrum use [18]. This line of work generally focuses only on sensing, not on a system for allocating spectrum based on sensing results. The work in [19] considers joint sensing and allocation, but assumes a model where Type 2 users are responsible for sensing and can be penalized for misreporting.

While our focus is on the design and robustness of the overall coordination system, techniques from the CR spectrum sensing literature may be useful in extending our work to cover probabilistic sensor fault models. In this case, we could apply existing data fusion techniques in the internal logic for how servers process sensor reports.

III. System and Threat Model

Our system consists of spectrum available for allocation, users, servers, and sensors with the following assumptions.

Spectrum. There exist F frequency bands F_i, $i = 1, 2, \cdots, \mathcal{F}$, where each band F_i may include S_i slices. To simplify discussion, we assume that bands are fungible (although this need not be true in general [20]) and users may receive a grant from any available band/slice.

Users. There are N_1 primary Type 1 users that have priority over N_2 secondary Type 2 users. Type 1 users do not make requests for spectrum access and may transmit at any time[1]. Their transmissions are detected by sensors that report the transmissions to the servers. In contrast, Type 2 users must request access from the servers and receive a *grant* response before beginning to transmit. A Type 2 user's request will be *denied* if it would interfere with an ongoing Type 1 or Type 2 transmission that the servers know about. A Type 2 user's grant will be *revoked* if the servers learn about a new Type 1 transmission with which the Type 2 transmission can interfere. We employ heartbearts and use lack of heartbeat to implement revocation (as opposed to explicit revocation messages) as a fail-safe mechanism: a Type 2 user who becomes disconnected from the servers (and cannot receive revocation commands) will not continue transmissions that should have been revoked and cause harmful interference. All users behave correctly: Type 2 users always issue requests before transmitting, only

[1]We assume that Type 1 users coordinate among themselves via an out-of-band mechanism such that there is no interference between Type 1 users. This coordination may be largely static (e.g. assigning a single Type 1 user to any particular spectrum slice or region) or may be more flexible (e.g. a trusted dynamic coordination subsystem operating only between Type 1 users.)

transmit upon receiving a grant response, and stop transmitting upon heartbeat expiration.

Servers. There are K servers, where each server maintains a database of ongoing Type 1 transmissions (detected via sensor reports) and Type 2 transmissions (explicitly requested and granted). Servers grant, deny, and revoke Type 2 users' spectrum allocations based on the current state and incoming user requests and sensor reports. Servers have accurate location and propagation models; i.e, for any requested or reported transmission, the server can accurately determine its interference with ongoing transmissions (if any). We consider a Byzantine fault model for servers, where f servers may be arbitrarily faulty out of $K \geq 3f + 1$ servers.

Sensors. M sensors are deployed over the relevant geographic area to detect Type 1 transmissions. We assume each sensor has the same sensing radius R_S. A given sensor m_i detects a Type 1 user u_j if the straight-line distance between m_i and u_j is less than or equal to R_S. We consider a Byzantine fault model, where g sensors may be arbitrarily faulty.

Cryptography. A public key infrastructure exists for each user, server, and sensor. Each can use its own private key to sign messages. Byzantine servers and sensors are computationally bounded, such that they cannot forge signatures.

Network and Timing. All communication with and between servers occurs out-of-band (e.g., using backhaul or wired links to the Internet). We assume a standard partial synchrony model for this communication, where there is an unknown Global Stabilization Time (GST), such that after GST, all transmissions between two correct servers arrive within a constant time Δ. In practice, as in other works (e.g. [21]), we do not require synchrony to hold forever after GST, but only guarantee that requests/reports are processed during "sufficiently long" periods of synchrony. We do not assume synchronized clocks, but do assume that the clocks of all users and servers run at roughly similar rates (bounded drift), such that the measurement of elapsed time is approximately the same at any two nodes. This is needed for reasoning about the duration of spectrum grants. It is also needed in many BFT SMR protocols for timeouts to be effective.

IV. System Architecture and Protocols

Our system is based on BFT SMR of the spectrum allocation servers. Servers accept requests from Type 2 users and reports from sensors and order these requests and reports using a BFT SMR protocol. Servers execute requests and reports sequentially in the order determined by the BFT SMR protocol, taking needed grant/deny/revoke actions upon execution. Any BFT SMR protocol that tolerates f Byzantine servers out of $3f + 1$ total servers in a partially synchronous network model may be used (e.g. PBFT [22], Hotstuff [21]). We collectively refer to user requests and sensor reports as *client updates* when the distinction between the two is not important.

A. Sensor Deployment Strategies

We consider two models for sensor deployment: *global* and *private*. In the global model, every server knows the identity and public key associated with each sensor, and vice versa. Every sensor can therefore send its detection reports to all servers, and all servers can authenticate the reports.

In the private model, sensors are partitioned into K disjoint cohorts, with each cohort assigned to exactly one server. In this model, each server only needs to know the identities and public keys of sensors in its own assigned cohort, and each sensor only needs the public key of its assigned server. Each sensor sends its reports only to its assigned server. The private model matches scenarios where multiple organizations deploy servers and sensors independently, such that sensors only communicate with servers deployed by the same organization (for simplicity, we assume each sensor communicates with exactly one server, but this can be generalized to multiple servers deployed by the same organization).

We describe protocols for coordination between users, sensors, and servers below and discuss the implications of these two sensor deployment models.

B. Protocols

User-Server. When a Type 2 user u wants to transmit, they construct a request of the form $(request, u, i, t, d)$, where u is the user's identifier, i is a user-specific sequence number, t is the timestamp of the user's local clock, and d is the duration for which the user requests spectrum, starting from time t. For example, a request with $t = 100, d = 100$ requests to transmit from the time the request is granted until time $t + d = 200$ on the user's local clock. The user signs their request using their private key and sends it to all servers. The user periodically retransmits their request until they collect a *certificate* of grant responses $(grant, u, i, t, d, F_i, s)$ or deny responses $(deny, u, i, s)$. A certificate includes $2f + 1$ matching responses from distinct servers s.[2] We assume users assign monotonically increasing sequence numbers to their requests and only have one in-flight request at a time (i.e. they must receive a response to one request before issuing a new one). This is a common assumption in BFT systems.

Upon collecting a certificate of $(grant, u, i, t, d, F_i, s)$ responses, the user may transmit in the assigned frequency band F_i until time $t + d$ on their local clock. A user who receives a certificate of deny responses may not transmit, but may choose to retry their request for a different time period t.

Users receive periodic heartbeats for an ongoing transmission i from servers of the form $(heartbeat, u, i, s, t_s)$, where s is the server ID and t_s is the local timestamp of server s. If a heartbeat timeout period T elapses in which the user has not received a new heartbeat from at least $2f + 1$ distinct servers s, it must stop transmission i. In this case, a *new* heartbeat from s is defined as one with a higher local timestamp t_s than any previously received heartbeat from s.

Sensor-Server. Each sensor m sends reports of the form $(report, m, i, L)$, where m is the sensor's identifier, i is a

[2]While $f + 1$ responses are enough to guarantee support from at least one correct server, requiring $2f + 1$ does not hurt availability ($2f + 1$ correct servers are needed in the BFT SMR protocol) and reduces attacker power ($f + 1$ Byzantine servers cannot generate arbitrary responses) [23].

sensor-specific sequence number, and L is a list containing an entry for each active Type 1 transmission currently detected by that sensor. Each entry contains location and signal strength information that allows servers to determine which ongoing or requested Type 2 transmissions would cause interference for that Type 1 transmission. After constructing a report, the sensor signs its report using its private key.

Each sensor sends its report periodically, as well as upon any change in the detection list, i.e., upon detecting that a new Type 1 transmission has started (adding it to the list) or an ongoing Type 1 transmission has ended (removing it from the list). Similarly to users, sensors assign monotonically increasing sequence numbers to their reports. However, sensors do not receive replies to their reports and may issue a new report at any time (possibly before their previous report has been processed). This is because sensor reports have "overtaken-by-events" semantics: each report is a snapshot of the current state, so our goal is for the servers to have the most up-to-date snapshot, rather than enforcing complete reliability for reports.

In the *global* model, each sensor sends its report to *all* servers. In the *private* model, each sensor sends the report only to its *assigned* server.

Servers: Introducing Updates. Upon receiving a client update, a server checks that the client-specific sequence number i is strictly higher than the last sequence number the server executed for that client and that the signature on the update is valid. If so, the server introduces it into the BFT SMR protocol for ordering. In the global model, signatures on both user requests and sensor reports can be verified by all K servers. In the private model, user requests can be verified by any server and directly injected into the BFT SMR protocol, but servers must take responsibility for reports on behalf of their sensors. For each report, the server appends its own identifier, signs the resulting message with its private key, and injects that message into the BFT SMR protocol for ordering.

If a server receives a retransmitted user request (sequence number less than or equal to its last executed sequence number for that user), and the transmission is active in the database (i.e. was granted and has not yet expired), it re-sends a *grant* response. Otherwise, it sends a *deny* response.

Servers: Executing Updates. Upon ordering an update, a server checks that its client-specific sequence number is higher than the last executed sequence number for this client. If not, it ignores the update.[3] Otherwise, the server executes it.
To execute a user request: the server checks whether there is an available frequency band F_i where the transmission will not interfere with any ongoing Type 1 or Type 2 transmissions based on its local database. If so, it issues a *grant* response assigning the user a slice in band F_i and records the transmission in its database. Otherwise, it issues a *deny* response.

[3]While this may seem redundant with the check at introduction time, BFT SMR protocols may not guarantee per-client FIFO ordering when sensors have multiple in-flight reports. Moreover, a duplicate update may be introduced during the time between introduction and ordering. Thus, checking at execution time is needed for correctness. Checking at introduction time is an optimization to avoid ordering unneeded updates.

To execute a sensor report: the server updates its local database with the contents of this report. It marks each Type 1 transmission included in this report as reported by sensor m in the database. For any Type 1 transmission that appears as reported by m in the database but is *not* included in this report, the server removes m from the list of reporting sensors for that transmission. For each Type 1 transmission included in the report, the server checks whether the *detection condition* is met, such that the report should be considered valid and acted on. Specifically, for the *global* sensor deployment model it checks if the transmission now has $g+1$ accumulated reports from different sensors, indicating that it was reported by at least one correct sensor. In the private model, it requires $g+1$ accumulated reports from different sensors *for each of* $f+1$ *servers* (a total of $(g+1) \times (f+1)$ reports). If so, the server checks whether it interferes with any ongoing Type 2 transmission(s) based on its local database. If so, it removes interfering Type 2 transmissions from its local database. After removal, the server will no longer send heartbeats for this transmission, revoking the allocation and requiring the user to stop transmitting. If removing m from the list of reporting sensors for a Type 1 transmission causes the number of reporting sensors to fall below $g+1$, no immediate action is needed, but that transmission will no longer be considered active when calculating interference for Type 2 requests.

Servers: Periodic Heartbeats. Each server periodically sends a heartbeat message $(heartbeat, u, i, s, t_s)$ for each active Type 2 transmission in its local database. There is no need to totally order heartbeat events through BFT SMR, as they do not change the server's state or affect its decisions for new requests. Each server sends heartbeats independently based on its local state, and users can transmit as long as they have sufficiently recent heartbeats from at least $2f+1$ servers.

Servers: Logical Timeouts. We use a *logical timeout protocol* (similar to [24]) to ensure all correct servers remove expired Type 2 allocations from their local databases at the same logical point in the total ordering of events. This is needed to avoid inconsistent decisions about new requests that arrive when some servers have already expired a potentially interfering transmission, but others have not.

Each server *locally expires* a Type 2 transmission when d time has passed on the server's local clock since executing the corresponding request, where d is the approved duration of the request. Assuming bounded clock drift, this ensures the server only expires the transmission *after* the user has already stopped transmitting (since the user calculates their end time based on the request *submission* time).

When a server locally expires a transmission, it introduces an update $(timeout, s, u, i)$ into the BFT SMR protocol for ordering. Upon executing $2f+1$ timeout updates from distinct servers s for the same transmission i from user u, a server removes the transmission's entry from its local database.

C. Discussion

At a high-level, our spectrum sharing system is a straightforward application of BFT SMR. However, fully specifying

the protocols showed that there are subtle design issues. For example, our initial idea for implementing sensor reports was for each sensor to send a separate report Type 1 transmission start and end event. We switched to the snapshot-style approach after realizing the complexity of handling edge cases in the separate report approach. If per-client FIFO ordering is not guaranteed with multiple outstanding requests, a transmission's end report may be processed *before* the corresponding start report. In this case, that start report should be ignored. However, other re-ordered start reports may still need to be either processed despite "old" timestamps or retransmitted with updated timestamps (which requires acknowledging reports to know when to stop retransmitting).

Similarly, our first approach to revocation was to use explicit *revoke* commands. However, this introduces the edge case of a temporarily disconnected user who does not receive the revoke command, which led us to the heartbeat approach. If heartbeat timeouts are relatively long, explicit revocations may still be sent as a performance optimization.

V. Evaluation

We assess the ability of our BFT spectrum sharing system to make effective spectrum allocation decisions under a range of attack scenarios through simulation.

A. Experimental Setup

1) Simulation: We develop a discrete-event simulation that models server-level and sensor-level adversaries. The simulation environment consists of a square region of side length D, within which N_1 Type 1 users, N_2 Type 2 users, and a total of M sensors are uniformly and independently distributed at random. The system includes K servers that coordinate spectrum allocation. In the *private* sensor-deployment model, the M sensors are evenly and randomly partitioned among the K servers, giving each server a sensor cohort of size $m = M/K$. Table I summarizes the simulation parameters.

We consider two spectrum bands with different access constraints. Band F_1 is a shared band in which both Type 1 and Type 2 users may be active; allocations in F_1 must avoid interfering with ongoing Type 1 transmissions. Band F_2 is a secondary-only band that is used exclusively by Type 2 users and does not require coordination with Type 1 activity. The two-band structure aids in protecting incumbent Type 1 transmissions in F_1 while providing access for Type 2 users in F_2 when F_1 is unavailable.[4]

The simulation operates in discrete time slots for a total of T slots. During each time slot, each Type 1 user follows a two-state Markov ON–OFF process, where $p_{\text{on}\to\text{off}}$ and $p_{\text{off}\to\text{on}}$ denote the transition probabilities between active and idle states. Each correct sensor detects and reports a given Type 1 transmission in the timeslot if it is within distance R_S from that Type 1 user. At the same time, each Type 2 user independently decides with probability P_{request}, whether to issue an allocation request. The duration requested for all Type 2 transmissions is L timeslots. We abstract the BFT SMR protocol by processing all reports and requests generated in a time slot in an arbitrary deterministic order, modeling the globally consistent order that SMR provides. For simplicity, we process all sensor reports in the timeslot first, and then all Type 2 user requests. This matches expected behavior if requests/reports are batched at the servers and has minimal impact otherwise; changing it will only cause some currently denied requests to be granted and then quickly revoked.

When processing each Type 2 request, a server selects a band to attempt (F_1 or F_2) pseudorandomly with equal probability 0.5. Within the chosen band, the server examines slices sequentially; it selects the first slice that passes the interference check, and *accepts* the request on that slice. If no available slice exists in the chosen band, the server repeats the search on the other band. If no slice is found in either band, the request is *denied*. A candidate slice is considered available for the requesting Type 2 user only if the replicated spectrum-usage state indicates (i) no active Type-1 transmitter

TABLE I: Simulation Parameters

Parameter	Value	Description
K	10	Number of servers.
$f_{\text{configuration}}$, f	3, varied	System-configured maximum number of Byzantine servers tolerated; Actual number of Byzantine servers.
N_1	10	Number of Type 1 users.
N_2	15	Number of secondary Type 2 users.
P_{request}	0.25	Probability a Type 2 issues a request per slot.
S_1, S_2 in F_1, F_2	1, 1	Number of spectrum slices in each band.
$p_{\text{on}\to\text{off}}$	0.25	Type 1 ON→OFF transition probability.
$p_{\text{off}\to\text{on}}$	0.50	Type 1 OFF→ON transition probability.
C_{attack}	{baseline, over-report, under-report}	Byzantine behavior type.
$P_{\text{byzantine}}$	$0 \to 1$	Probability that a compromised node performs its attack in a given slice when reporting.
T	10,000	Total time slots simulated.
L	50	Duration (in slots) of each Type 2 allocation.
D	1000	Side length of square region (m).
R_{T1}	300	Type 1 interference radius (m).
$R_{T2}^{F_1}$	250	Type 2 interference radius in F_1 (m).
$R_{T2}^{F_2}$	150	Type 2 interference radius in F_2 (m).
R_S	200	Sensor sensing radius (m).
M	varied	Total number of sensors.
$m = M/K$	varied	Number of sensors per server.
$g_{\text{configuration}}$, g	varied	System-configured maximum number of Byzantine sensors per cohort; Actual number of Byzantine sensors per cohort.

[4]CBRS bands are often used by service providers to offload traffic from other licensed spectrum, knowing that they may have to evacuate the CBRS bands if a US Navy radar transmission is detected in that band. Evacuated users may be assigned resources in other licensed spectrum (e.g., C-Band adjacent to CBRS or even the 600 MHz bands).

lies within the interference radius R_{T1} of the Type 2 user, and (ii) no previously admitted Type-2 user on the same band lies within distance $R_{T2}^{F_1}$ or $R_{T2}^{F_2}$ of the requester, depending on whether the slice belongs to the band F_1 or F_2. Upon acceptance, the allocation remains active for L slots. If, at a later time, a newly detected Type 1 transmission conflicts with an ongoing Type 2 allocation, the allocation is *revoked*.

The simulation faithfully represents our system with some simplifying assumptions: transmission times align to timeslots, sensors report periodically each timeslot, and detection/coordination/processing delays are negligible. In practice, delays may cause brief transient interference between when a Type 1 transmission starts and when interfering Type 2 users learn of revocation, but such delays should be minimal given wired backhaul coordination. This transient interference is separate from our main focus: how Byzantine behavior affects allocation decisions and system performance (utilization and interference), and the server/sensor count needed to mask such behavior.

2) Attack Models: We evaluate two Byzantine behaviors:

1) **Over-report**: A Byzantine node falsely reports activity that does not actually exist. For sensors, this means reporting a Type 1 transmission that does not exist. For servers, this includes reporting a non-existent Type 1 transmission on behalf of its sensors (in the private model) or falsely recording that a Type 1 or Type 2 user is active in a slice and generating denial responses based on this incorrect local state (in private and global models).
2) **Under-report**: A Byzantine node does not report activity that is actually present. For sensors, this means failing to report an active Type 1 transmission. For servers, this means not reporting a Type 1 transmission detected by its sensors (in the private model) or failing to record an active Type 1 or Type 2 transmission and generating grant responses based on this incorrect local state (in private and global models).

In the worst case, compromised nodes attack deterministically: always *over-reporting* to maximize Type 2 denials, or always *under-reporting* to maximize collisions and degrade useful service for all users. However, we explore the impact of a range of malicious behavior by parameterizing attack intensity using a probability $P_{\text{byzantine}}$. For each sensor reporting event and each server reporting or decision-making event, a Byzantine node applies its attack behavior with probability $P_{\text{byzantine}}$, randomizing for each involved spectrum slice. $P_{\text{byzantine}} = 1$ represents deterministic malicious behavior.

3) Metrics: To quantify the effect of Byzantine behavior on the systems ability to maintain Type 2 service availability while protecting Type 1 incumbents from interference, we evaluate four performance metrics:

- **Utilization**: The fraction of slice-time slots[5] that are used without conflicts (i.e., occupied by Type 1 and/or Type 2 users without overlapping interference ranges).
- **Denial Rate**: The fraction of Type 2 requests that are denied because no slice can be allocated without conflicts.
- **Collision Rate**: The fraction of slice-time slots that experience interference conflicts, either between (1) Type 1 and Type 2 users within interference range, or (2) multiple Type 2 users within interference range.
- **Revocation Rate**: The fraction of slice-time slots where an active Type 2 allocation is terminated upon detection of a Type 1 user within interference range.

[5]This is a time slot t in slice F_i

B. Impact of Server Compromises

If the actual number of compromised servers f exceeds the configured $f_{\text{configuration}}$ threshold for the number of tolerated compromises, the system we describe does not provide any guarantees. But, the *practical impact* depends on the type of attack and on application characteristics that allow some functionality *even when coordination breaks down*.

In the following two experiments, we fix $f_{\text{configuration}} = 3$, which yields $K = 3f_{\text{configuration}}+1 = 10$ servers. We assume 200 sensors deployed under a private model and assume all sensors are correct ($g_{\text{configuration}} = g = 0$). We sweep the actual number of compromised servers f from 0 to 10, and vary the attack activation probability $P_{\text{byzantine}}$ from 0 to 1. When $P_{\text{byzantine}} = 1$, a compromised server behaves fully adversarially and always performs the attack on each slice; when $P_{\text{byzantine}} = 0$, compromised servers behave like correct servers. Server-level compromises corrupt the replicated spectrum-usage state for both Type 1 detection and Type 2 allocation on both bands, so we report aggregate performance without separating bands.

Figure 1(a) shows the effect of *over-report* attacks. When $f \leq f_{\text{configuration}}$, aggregate utilization remains near 1 and the collision rate remains 0, demonstrating that the system maintains correct operation under the configured fault threshold. Once $f > f_{\text{configuration}}$ (here, when $f = 4$), denials begin to rise, as compromised servers incorrectly report slices in band F_1 as being occupied by Type 1 users. However, this does not have a noticeable effect on utilization, as Type 2 users continue to successfully transmit on band F_2. For $P_{\text{byzantine}} = 1$, once the number of compromised servers reaches $2f_{\text{configuration}} + 1$ (here, $f = 7$), the denial rate goes to 1 and utilization drops to 0.5, as Byzantine servers can unilaterally deny requests from Type 2 users, preventing all use of band F_2. Type 1 users continue to transmit in band F_1 without interference.

Figure 1(b) shows that while *under-report* attacks only have visible effects at higher f values, they can have an even more severe impact on utilization. When f first exceeds the configured threshold $f_{\text{configuration}} = 3$, there is essentially no effect: utilization remains high, denial rates stay nearly unchanged, and collision rates remain close to zero. Even though the theoretical bound is violated, there remain at least $f_{\text{configuration}}+1$ servers reporting most Type 1 transmissions and protecting Type 2 allocations. However, at $f = 6$, we see denials begin to decrease, causing collisions to increase and utilization to decrease. At this point, we still have $f + 1 = 4$ correct servers, but due to sensing limitations, there are cases

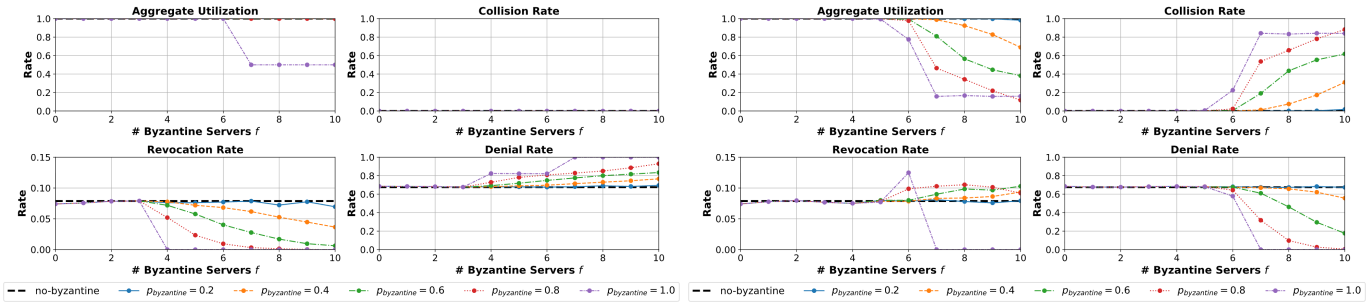


Fig. 1: Performance of the BFT-based spectrum sharing system under server-level Byzantine attacks. **(a) Over-report attacks. (b) Under-report attacks**.

where we do not have $f + 1$ correct servers that *have sensors in range* of every Type 1 transmission, so some active transmissions go undetected. At $f \geq 7$, Byzantine servers can unilaterally grant Type 2 requests, leading to a collision rate of over 0.8, and corresponding aggregate utilization less than 0.2, as *all* Type 2 users believe they are allowed to transmit (regardless of the interference they cause).

Our simulation only focuses on *application-level* misbehavior: because our approach is agnostic to the specific BFT SMR protocol used, we do not consider attacks on the replication protocol itself. However, the extreme conditions of $P_{\text{byzantine}} = 1$, $f \geq 2f_{\text{configuration}} + 1$ capture the worst-case behavior even with such attacks: in these conditions, Byzantine servers can issue *arbitrary* deny responses (over-report case) or grant responses (under-report case). Thus, the extreme over-report case is a full denial of service attack on all Type 2 users, equivalent to denying or simply not processing their requests. In the extreme under-report case, all requests are granted, which is equivalent to a system where all Type 2 users attempt to transmit with no coordination. Attacks on the BFT SMR protocol may be able to make these conditions occur earlier (at $f_{\text{configuration}} + 1$ instead of $2f_{\text{configuration}} + 1$), but the results when they do occur will be the same.

Our results show that the impact of server-level attacks depends not only on the number of server faults the system is configured to tolerate, but also on the characteristics of the spectrum sharing application, and physical sensor deployment. First, we see that diversity in coordination mechanisms and band access provides some inherent resilience. Because Type 1 users employ out-of-band coordination, their transmissions are not affected in the over-report case. Moreover, because band F_2 is *only* used by Type 2 users, it is not affected in the under-report condition for $f_{\text{configuration}} < f < 2f_{\text{configuration}} + 1$, where Byzantine servers can misrepresent Type 1 behavior but not arbitrarily grant access to Type 2 users. However, we still see that malicious server behavior beyond the $f_{\text{configuration}}$ threshold has a large impact on utilization and collisions, further motivating the use of Byzantine fault tolerance.

Moreover, we see that the boundary at which *under-report* becomes harmful is determined not only by the configured thresholds $(f_{\text{configuration}}, g_{\text{configuration}})$, but also by sensor deployment characteristics. Any given server may fail to report a Type 1 transmission either because (i) its local sensor reports of that Type 1 transmission fail to reach the $g_{\text{configuration}}+1$ threshold, or because (ii) the server withholds the detection despite receiving sufficient reports from its sensor cohort. Thus, interference protection depends on whether the sensor deployment provides enough overlapping coverage such that, for each active Type 1 transmission, more than $f_{\text{configuration}}+1$ servers each receive and report at least $g_{\text{configuration}}+1$ sensor detections. BFT replication does not itself ensure a Type 1 transmission is detected; it only ensures agreement on the reports. Safety also depends on whether the sensing layer supplies sufficient evidence of Type 1 activity. While dense sensor deployment increases the likelihood of detection, it also increases cost and deployment overhead.

This naturally raises the next question: *how dense must the sensor deployment be to guarantee that Type 1 transmissions remain detectable under Byzantine behavior?* We address this in Sections V-C and V-D.

C. Guaranteeing Detection of Type 1 Transmissions

We now examine how sensor deployment density influences the system's ability to provide reliable Type 1 detection under Byzantine behavior. We evaluate four deployment models that differ in how sensors are assigned to servers and whether sensors may be Byzantine. In this section, we determine, for each model, the sufficient worst-case sensing "density" needed to ensure that an adversary who controls up to f servers and g sensors cannot prevent a Type 1 transmission from being detected. Here, we assume the configured fault tolerance thresholds are not exceeded, i.e. $f = f_{\text{configuration}}$ and $g = g_{\text{configuration}}$, since the goal is to determine the sensing density required for a system with a fixed Byzantine threshold.

As discussed in Section IV-A, we consider two sensor deployment choices: *global* and *private*. These two choices, together with whether sensors may be Byzantine, produce four deployment models. For each model, we state its *detection requirement* in terms of how many sensors need to be in range of a given Type 1 user in order to detect its transmission:

1) **Model A (Global, Fully-Trusted).** All sensors are assumed correct ($g_{\text{configuration}} = 0$). A Type 1 transmission is detected as long as *at least one* sensor is in range. *Detection requirement:* at least 1 sensor in range.
2) **Model B (Global, Untrusted).** Up to $g_{\text{configuration}}$ sensors may be Byzantine. A Type 1 transmission is "detected"

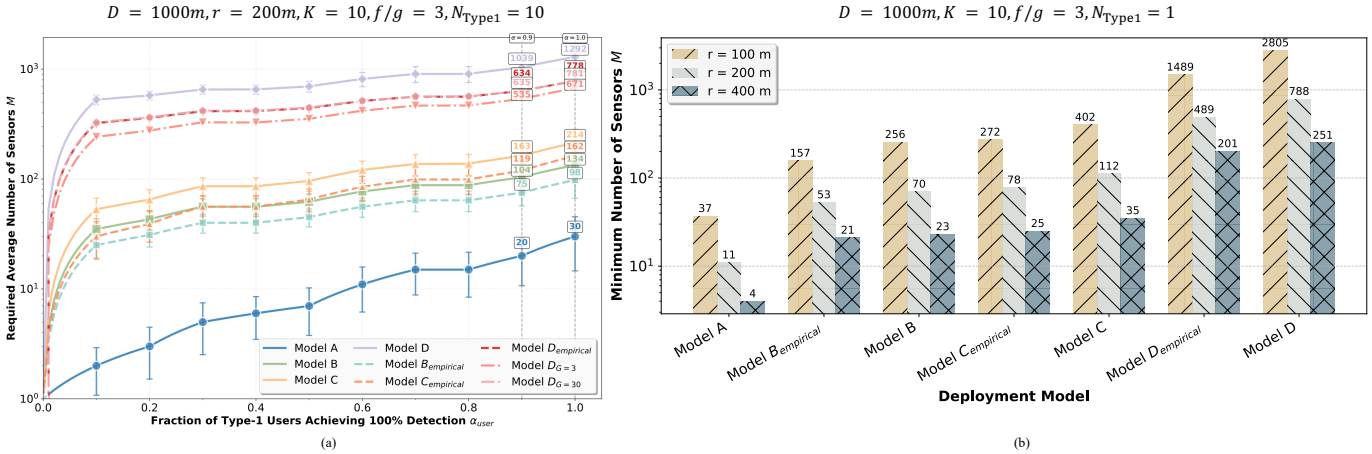


Fig. 2: The required average number of sensors M needed to guarantee Type 1 detection across different deployment models. **(a)** is the average M needed to detect a fraction of Type 1 users in each trial (averaged over trials). **(b)** is the average M required to guarantee one randomly placed Type 1 user is detected, shown for different sensing ranges R_S.

only if at least $g_{\text{configuration}}+1$ sensors report the transmission. Thus, each Type 1 transmission must be in range of at least $2g_{\text{configuration}}+1$ sensors to guarantee correct detection (since g sensors may under-report).
Detection requirement: at least $2g_{\text{configuration}}+1$ sensors in range.

3) **Model** C **(Private, Fully-Trusted).** All sensors are assumed correct ($g_{\text{configuration}} = 0$). A server reports a Type 1 transmission if any sensor in its cohort reports it. A Type 1 transmission is "detected" if $f+1$ servers report it. Thus, at least $2f_{\text{configuration}}+1$ server cohorts must have a sensor that reports the transmission to guarantee detection.
Detection requirement: at least 1 sensor in each of at least $2f_{\text{configuration}}+1$ server cohorts.
4) **Model** D **(Private, Untrusted).** Up to $g_{\text{configuration}}$ sensors per cohort may be Byzantine. Each cohort requires $2g_{\text{configuration}}+1$ sensors in range to ensure $g_{\text{configuration}}+1$ reports of the Type 1 transmission, and this must occur in at least $2f_{\text{configuration}}+1$ servers to ensure at least $f_{\text{configuration}}+1$ correct servers report it.
Detection requirement: at least $2g_{\text{configuration}}+1$ sensors in each of at least $2f_{\text{configuration}}+1$ server cohorts.

To quantify the minimum sensing density required in the worst-case situation under each model, we perform 5000 independent Monte Carlo trials. In each trial, we:

1) Place N_1 Type 1 users uniformly at random in the $D \times D$ region.
2) Deploy sensors sequentially at random positions, marking any Type 1 users within the sensing range R_s of a particular sensor as "detected" by that sensor.
3) After each placement, check whether the detection requirement for each Type 1 user is met according to the criterion defined above for each Model $A, B, C,$ and D.
4) Stop once the detection condition is satisfied for all Type 1 users and record: the minimum number of sensors required to detect (i) all Type 1 users, and (ii) at least a target fraction α_{user} of Type 1 users.

As shown in Figure 2(a), the four deployment models form a natural progression in detection difficulty. Trust assumptions weaken from Model A to B and from Model C to D, while the sensing architecture becomes more constrained when moving from the global models (A, B) to the private models (C, D). Model D combines both the weakest trust assumption and the most restrictive sensing architecture, making it the most pessimistic and the most demanding scenario. Model D requires nearly an order of magnitude more sensors to guarantee 100% Type 1 user detection than the other models: it requires close to 1300 sensors, whereas Models B and C require only tens to low hundreds, an 85–90% reduction under the same conditions, and Model A, the baseline, requires only about 30 sensors.

These results underscore that worst-case guarantees regarding Byzantine behavior come at an extremely high cost. In Model D, the system must deploy nearly 1300 sensors, roughly 130 per server, while tolerating only 3 Byzantine sensors per cohort, to guarantee all Type 1 transmissions are detected. However, the property it provides is very strong: for *every* Type 1 user, even if an attacker can compromise 3 servers (rendering *all* of their assigned sensors, approximately 390 total, useless) and can compromise the 3 closest sensors to that user in each of the remaining 7 cohorts (21 total sensors), there will still be 4 correct sensors in each of the 7 cohorts (28 total) that detect the transmission.

D. Detecting Type 1 Transmissions in Practice

Because the worst-case analysis above assumes a very strong attacker, we also consider a weaker attacker who can compromise f servers and g sensors per cohort, but cannot deterministically choose the servers and sensors to perfectly target a given Type 1 user. Instead, we assume compromised servers and sensors are randomly selected (modeling a scenario where an attacker cannot exploit *any* node they choose but may only have access to or exploits for a subset of nodes).

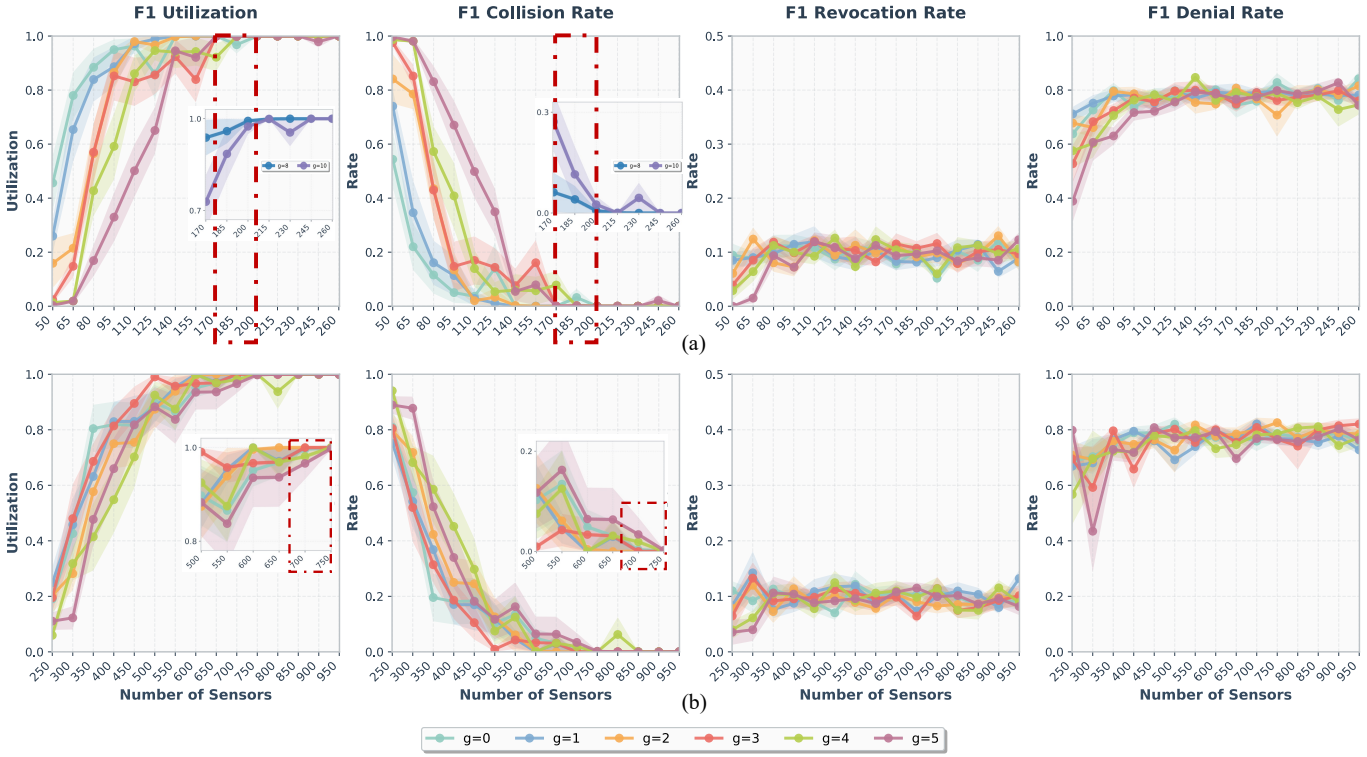


Fig. 3: Evaluating the influence of sensor deployment density on system performance under different sensor-trust configurations. The Byzantine behavior is the special case of *under-report*, $P_{\text{byzantine}} = 1$. We report metrics only for band F_1, as F_2 is not affected by sensor under-reporting. **(a)** $g_{\text{configuration}} = 0$ **(b)** $g_{\text{configuration}} = 3$

Thus, we evaluate *empirical* variants of Models B, C, and D. The approach is similar to the one above, but we simulate random compromises and evaluate whether each Type 1 user has a sufficient number of *correct* sensors in-range for it to be detected. Specifically, the detection criteria for a given Type 1 transmission in each model are:

1) **Model $B_{\text{empirical}}$:** at least $g_{\text{configuration}}+1$ *correct* sensors are in range.
2) **Model $C_{\text{empirical}}$:** at least 1 correct sensor per-cohort for each of at least $f_{\text{configuration}}+1$ correct servers is in range.
3) **Model $D_{\text{empirical}}$:** at least $g_{\text{configuration}}+1$ correct sensors per-cohort for each of at least $f_{\text{configuration}}+1$ correct servers are in range.

We perform 5000 independent Monte Carlo trials; in each:

1) We sample N_1 Type 1 users uniformly at random in a $D \times D$ region.
2) Using reservoir sampling, we uniformly select up to $g_{\text{configuration}}$ sensors in each server's cohort (under *private* models) or among all sensors (under the *global* models) as Byzantine; all remaining sensors are correct. We randomly designate $f_{\text{configuration}}$ servers as Byzantine.
3) We add sensors sequentially at random positions, marking any Type 1 user within the sensing range R_s of a particular sensor as "detected" by that sensor.
4) We stop sensor deployment once the corresponding model-specific detection requirement defined above (Models $B_{\text{empirical}}$, $C_{\text{empirical}}$, and $D_{\text{empirical}}$) is satisfied for all Type 1 users.
5) We record the number of sensors required to detect (i) *all* Type 1 users and (ii) the number of sensors required to detect at least the target fraction α_{user} of them.

Figure 2(a) shows that the empirical sensor deployment models require fewer sensors than their worst-case counterparts. In particular, Model $D_{\text{empirical}}$ requires nearly *50% fewer* sensors than the worst-case Model D to achieve full Type 1 detection, while Models $B_{\text{empirical}}$ and $C_{\text{empirical}}$ reduce the required sensor count by roughly *25%* relative to Models B and C. These results demonstrate that smaller numbers of sensors are likely to be effective in most cases, although they do not provide strict guarantees.

We additionally evaluate two variants derived from the Model $D_{\text{empirical}}$. The empirical models above assume that up to $g_{\text{configuration}}$ sensors may be compromised *per cohort*, yielding a total of $G = g_{\text{configuration}} \cdot K$ compromised sensors (i.e., 30 in our setting). To isolate the effect of this per-cohort constraint, we introduce two additional variants:

1) **Model $D_{\text{G=30}}$:** The total number of compromised sensors remains $g_{\text{configuration}} \cdot K$ (e.g., 30), but they are sampled uniformly at random over *all* sensors rather than being constrained to $g_{\text{configuration}}$ per cohort.
2) **Model $D_{\text{G=3}}$:** Only $g_{\text{configuration}}$ total sensors are compromised (i.e., global $G = 3$ instead of 30). These G Byzantine sensors are selected uniformly at random from all deployed sensors, regardless of cohort boundaries.

Figure 2(a) shows that Model $D_{\text{G=30}}$ yields sensing requirements that are nearly identical to those of Model $D_{\text{empirical}}$. Once the total adversarial budget is fixed, the *distribution* of corrupted sensors across cohorts has little influence on the needed sensing density. While Model $D_{\text{G=3}}$ reduces the total number of compromised sensors from 30 to only 3, it decreases the required sensing density by only about 15% relative to Model $D_{\text{empirical}}$. This suggests that the dominant factor driving the sensing requirements in Model D is not the number of Byzantine sensors, but the per-cohort requirement that *each* server must obtain at least $g_{\text{configuration}}+1$ correct in-range reports. Even a tenfold reduction in the number of corrupted sensors translates into only a modest reduction in the total sensing density needed to guarantee full Type 1 detection.

Our results assume a sensing radius $R_s = 200\,\text{m}$. However, sensing range has a substantial effect on the number of sensors required. As shown in Figure 2(b), increasing the sensing radius from $100\,\text{m}$ to $200\,\text{m}$ reduces the required sensor count by roughly 60–70%. Extending it further to $400\,\text{m}$ yields an overall reduction of about 85–90% across all deployment models. However, in practice, increasing sensing radius is not a simple solution to reduce the number of required sensors: increasing range requires more sensitive sensors, which are more expensive, and, more critically, are more susceptible to spurious detections (detecting a Type 2 user's signal and assuming it is a Type 1 transmission).

E. Application Impacts of Byzantine Sensor Behavior

Having analyzed the sensing density required to guarantee Type 1 detectability, we now explore the application-level impacts of sensors failing to detect Type 1 transmissions, or falsely reporting non-existent transmissions. We restrict our analysis to the *private* deployment model and assume $f = 0$ to focus on the effect of sensor-level adversaries (although we keep $f_{\text{configuration}} = 3$, as in the previous sections). We evaluate how sensor misbehavior affects the four metrics defined in Section V-A: (i) slice utilization, (ii) collision rate, (iii) revocation rate, and (iv) denial rate.

First, we evaluate behavior when the system is *not* configured to tolerate Byzantine sensor behavior (i.e. $g_{\text{configuration}} = 0$). To study the impact of Byzantine under-reporting, we vary the *actual* number of compromised sensors $g \in \{0, 1, \ldots, 5\}$ per cohort. We focus exclusively on under-reporting here because, with $g_{\text{configuration}} = 0$, the system is inherently vulnerable to over-reporting: even a single fabricated report will cause servers to incorrectly consider a slice as occupied.

As shown in Figure 3(a), system performance becomes largely insensitive to under-reporting once the number of sensors exceeds roughly 200: slice utilization approaches 1.0 and the collision rate falls to nearly zero across all $g \in 0, \ldots, 5$. This aligns with the result for Model $C_{\text{empirical}}$ in Figure 2(a), which shows 162 sensors are sufficient to detect all Type 1 transmissions; collisions go to zero for $g = 0$ at around 150-160 sensors and for the other g settings by about 200 sensors. The inset plots further show that the same robustness persists even for $g = 8$ and $g = 10$, corresponding to *close to half of the sensors in each cohort being Byzantine*. Under dense deployments, almost every Type 1 user is guaranteed to have at least one correct sensor (for at least $f+1$ servers) in range, effectively neutralizing the impact of under-reporting even when the number of compromised sensors is substantially higher than the system was configured to tolerate. Moreover, neither the revocation rate nor the denial rate exhibits noticeable degradation as g increases, reinforcing that under-reporting alone has limited power to disrupt allocation decisions when sensing coverage is sufficiently high.

Next, we set $g_{\text{configuration}} = 3$, matching the setting used in Sections V-C and V-D. This threshold requires each server's cohort to provide at least four in-range reports before counting a Type 1 transmission as detected, thereby protecting against both under-report and over-report behavior. In Figure 3(b), we observe that the overall performance curves largely retain the same qualitative trend as in the baseline case ($g_{\text{configuration}} = 0$). However, the sensor density required to reach the same performance level increases substantially. For example, in the baseline configuration, deploying around 200 sensors is already sufficient to tolerate up to $g = 5$ under-reporting Byzantine sensors without noticeable degradation. In contrast, under the stricter configuration $g_{\text{configuration}} = 3$, the system now requires on the order of 900 sensors, nearly a fivefold increase, to achieve comparable utilization, collision, denial, and revocation rates.

The inset in Fig. 3(b) shows a threshold behavior at the tolerated $g_{\text{configuration}}$ value: once the total number of sensors reaches roughly 700–750, the system achieves near-perfect utilization (≈ 1.0) and zero collisions for all cases with $g \leq 3$, whereas performance begins to degrade when $g \geq 4$. This means that a deployment in this density range provides exactly the level of robustness targeted by the configuration, sufficient yet not over-provisioned. This also aligns with the result from Figure 2, showing around 780 sensors were sufficient to detect all Type 1 users under Model $D_{\text{empirical}}$.

Based on this observation, we fix the deployment at 700 sensors and perform an exploratory analysis of how performance degrades as the actual number of compromised sensors increases. We sweep the true number of Byzantine sensors g from 0 to 10 and vary the attack-activation probability $P_{\text{byzantine}}$ from 0 to 1. Unlike the previous results, which are based on repeated Monte Carlo trials, these exploratory results are based on only a single random seed, so the exact values of these metrics should not be considered definitive. However, informal experimentation with multiple seeds suggests that while absolute values can vary significantly, the overall outcomes are consistent. The results are shown in Figure 4.

For $g \leq g_{\text{configuration}}$, we see negligible performance impacts, as expected. Beyond the configured threshold, system performance can vary based on the random seed, but in this case remains stable (high utilization, negligible collisions) at $g = 4, 5, 6$. Here, once $g > 6$, we observe an impact of sensor-level *under-reporting* on overall system performance: utilization decreases (here to about 0.7), and collision rate increases (here to about 0.3). Over-reporting continues to have minimal

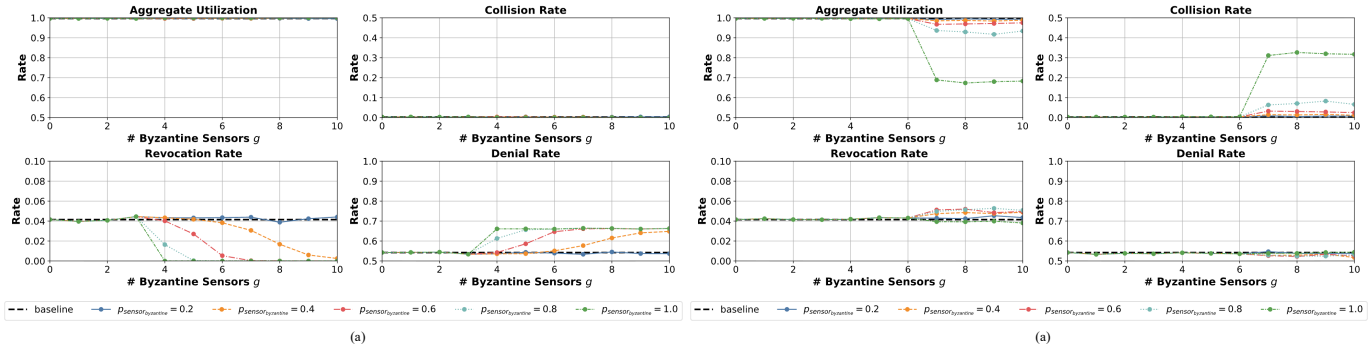


Fig. 4: Performance of the BFT-based spectrum sharing system under sensor-level Byzantine attacks, when $g_{\text{configuration}} = 3$. **(a) Over-report attacks. (b) Under-report attacks**. These results use a single random seed = 2147483647.

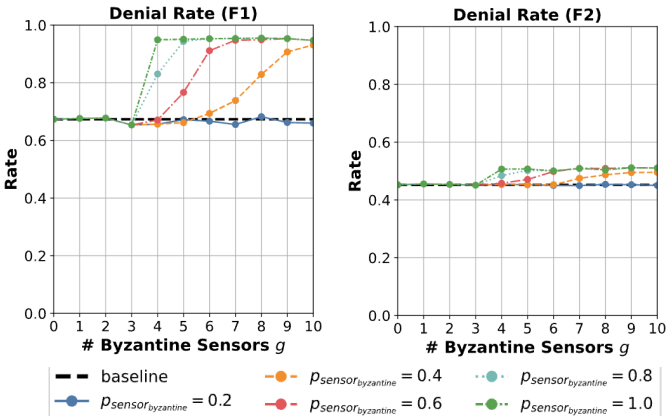


Fig. 5: Separate denial rate performance for the two bands $F1$ and $F2$ for the experiment shown in Figure 4(a).

impact. In contrast, server-level attacks result in collision rates that make the system essentially unusable (Figure 1). Unlike servers, sensors cannot arbitrarily grant or deny requests; they can only influence decisions by misreporting Type 1 activity.

At our chosen level of sensor density, there is enough redundancy that collisions remain low, even when 4-5 sensors per cohort under-report Type 1 transmissions. Over-reporting attacks are easier to execute: 4 Byzantine sensors (per cohort) can always convince servers that an idle slice is occupied, which shows up as a sharp increase in denial rate at $g = 4$. However, while this over-reporting reduces the spectrum available to Type 2 users, it has a low impact on overall utilization: Type 1 users continue to transmit normally, and Type 2 users continue to transmit in their dedicated F_2 band. Figure 5 further shows how the F_2 band helps maintain overall utilization. Compromised sensors can cause denials to go to 1.0 in the F_1 band, completely preventing Type 2 users from using this spectrum. This causes denials to also increase in the F_2 band, as it increases contention, but that spectrum remains available to Type 2 users.

This highlights an inherent trade-off in protecting against over-report vs under-report behavior. Increasing $g_{\text{configuration}}$ increases the bar for an attacker to execute a successful over-report attack, requiring them to compromise $g_{\text{configuration}} + 1$ sensors in order for a malicious report to be believed. However, somewhat counterintuitively, increasing the tolerated number of sensor compromises *increases vulnerability to under-report attacks*. Because we require $g_{\text{configuration}} + 1$ matching reports before considering a transmission detected, increasing $g_{\text{configuration}}$ increases the total number of sensors that must be in range of a transmission in order for it to be detected. Thus, it increases the likelihood that compromising some sensors will cause detection reports to fall below the required threshold for some Type 1 transmissions.

Therefore, the ideal $g_{\text{configuration}}$ value depends on the characteristics of the specific spectrum sharing application: if protection of Type 1 users from interference is the most important goal, a lower $g_{\text{configuration}}$ value (or much larger investment in sensing infrastructure) may be needed. If the goal is to maximize use of the available spectrum, a higher $g_{\text{configuration}}$ value offers better protection against malicious over-reporting. It is likely possible to use application-specific knowledge to reduce over-reporting vulnerability while keeping $g_{\text{configuration}}$ low (and thus achieve good under-reporting protection at reasonable sensor cost). For example, if sensor locations are known to the servers, they can filter malicious reports of Type 1 activity outside a given sensor's sensing radius.

VI. Conclusion

We have presented a BFT spectrum sharing system that tolerates Byzantine behavior of both servers and sensors and analyzed its robustness to malicious behavior through simulation. Our results provide insights that can help guide the deployment of future dynamic spectrum sharing schemes. We show that guaranteeing full masking of under-reporting and over-reporting behavior can be costly in terms of the number of required sensors, especially in private sensor deployment models where each server only receives direct reports from its "own" sensors. We highlight the tension between over-report and under-report protection: requiring more matching sensor reports increases robustness to over-reporting (allows efficient spectrum usage), but increases vulnerability to under-reporting (causes interference to incumbents). Application needs, especially whether incumbents can tolerate interference will be a critical design decision.

REFERENCES

[1] A. Abouzeid *et al.*, "National spectrum strategy research and development plan," *National Science and Technology Council*, October 2024.

[2] WinnForum, "List of SAS administrators," *https://cbrs.wirelessinnovation.org/sas-administrators*, 2024.

[3] M. B. H. Weiss, K. Werbach, D. C. Sicker, and C. E. C. Bastidas, "On the application of blockchains to spectrum management," *IEEE Transactions on Cognitive Communications and Networking*, vol. 5, no. 2, pp. 193–205, 2019.

[4] L. Perera, P. Ranaweera, S. Kusaladharma, S. Wang, and M. Liyanage, "A survey on blockchain for dynamic spectrum sharing," *IEEE Open Journal of the Communications Society*, vol. 5, pp. 1753–1802, 2024.

[5] The Federal Communications Commission, "In the matter of lower 37 GHz band: Use of spectrum bands above 24 GHz for mobile radio services," *Sixth Report and Order and Notice of Proposed Rulemaking: GN Docket No. 14-177*, April 29, 2025.

[6] (2024, May) Open afc - telecom infra project. [Online]. Available: https://telecominfraproject.com/open-afc/

[7] Y. Xiao, S. Shi, W. Lou, C. Wang, X. Li, N. Zhang, Y. T. Hou, and J. H. Reed, "Bd-sas: Enabling dynamic spectrum sharing in low-trust environment," *IEEE Transactions on Cognitive Communications and Networking*, vol. 9, no. 4, pp. 842–856, 2023.

[8] M. Grissa, A. A. Yavuz, and B. Hamdaoui, "Trustsas: A trustworthy spectrum access system for the 3.5 ghz cbrs band," in *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications*, 2019, pp. 1495–1503.

[9] T. Ariyarathna, P. Harankahadeniya, S. Isthikar, N. Pathirana, H. D. Bandara, and A. Madanayake, "Dynamic spectrum access via smart contracts on blockchain," in *2019 IEEE Wireless Communications and Networking Conference (WCNC)*, 2019, pp. 1–6.

[10] Z. Chen, L. Wang, and Y. Zhang, "Blockchain structure electromagnetic spectrum database in distributed cognitive radio monitoring system," *IEEE Transactions on Cognitive Communications and Networking*, vol. 8, no. 4, pp. 1647–1664, 2022.

[11] Z. Cheng, Y. Liang, Y. Zhao, S. Wang, and C. Sun, "A multi-blockchain scheme for distributed spectrum sharing in cbrs system," *IEEE Transactions on Cognitive Communications and Networking*, vol. 9, no. 2, pp. 266–280, 2023.

[12] Z. Li, W. Wang, Q. Wu, and X. Wang, "Multi-operator dynamic spectrum sharing for wireless communications: A consortium blockchain enabled framework," *IEEE Transactions on Cognitive Communications and Networking*, vol. 9, no. 1, pp. 3–15, 2023.

[13] K. Yan, W. Ma, S. Sun, and W. Wang, "Blockchain-based dynamic spectrum sharing for service-centric 6g networks: An evolutionary approach," *IEEE Transactions on Network Science and Engineering*, vol. 13, pp. 485–500, 2026.

[14] S. T. Muntaha, P. I. Lazaridis, M. Hafeez, Q. Z. Ahmed, F. A. Khan, and Z. D. Zaharis, "Blockchain for dynamic spectrum access and network slicing: A review," *IEEE Access*, vol. 11, pp. 17 922–17 944, 2023.

[15] L. Zhang, G. Ding, Q. Wu, Y. Zou, Z. Han, and J. Wang, "Byzantine attack and defense in cognitive radio networks: A survey," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 3, pp. 1342–1363, 2015.

[16] K. Marzullo, "Tolerating failures of continuous-valued sensors," *ACM Trans. Comput. Syst.*, vol. 8, no. 4, p. 284–304, Nov. 1990. [Online]. Available: https://doi.org/10.1145/128733.128735

[17] C. Jin, C. Yin, M. van Dijk, S. Duan, F. Massacci, M. K. Reiter, and H. Zhang, "Pg: Byzantine fault-tolerant and privacy-preserving sensor fusion with guaranteed output delivery," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 3272–3286. [Online]. Available: https://doi.org/10.1145/3658644.3670343

[18] A. Mollakhani and D. Guo, "Fault-tolerant spectrum usage consensus for low-earth-orbit satellite constellations," 2024. [Online]. Available: https://arxiv.org/abs/2312.05213

[19] J. Gan, J. Wu, P. Li, Z. Chen, J. Zhang, Z. Chen, J. He, and S. Fan, "Joint spectrum sensing and resource allocation against byzantine attack in overlay cognitive radio networks," *Transactions on Emerging Telecommunications Technologies*, vol. 33, no. 7, p. e4500, 2022.

[20] M. Weiss, P. Krishnamurthy, L. E. Doyle, and K. Pelechrinis, "When is electromagnetic spectrum fungible?" in *2012 IEEE International Symposium on Dynamic Spectrum Access Networks*, 2012, pp. 349–357.

[21] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, "Hotstuff: Bft consensus with linearity and responsiveness," in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, ser. PODC '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 347–356. [Online]. Available: https://doi.org/10.1145/3293611.3331591

[22] M. Castro and B. Liskov, "Practical byzantine fault tolerance and proactive recovery," *ACM Transactions on Computer Systems*, vol. 20, no. 4, pp. 398–461, 2002.

[23] J. Li and D. Mazieres, "Beyond one-third faulty replicas in byzantine fault tolerant systems." in *NSDI*, 2007, pp. 10–10.

[24] J. Kirsch, *Intrusion-tolerant replication under attack*. The Johns Hopkins University, 2010.